

Time-Series Power Flow — User Guide

This guide walks you through running a time-series power flow study from scratch: building a circuit, preparing the input CSV, running the simulation, and working with the results.

The `TimeSeriesPowerFlow` class in `timeseries.py` wraps the Newton-Raphson solver and runs it repeatedly across time steps. At each step, load values are scaled by multipliers you supply in a CSV file. Bus voltages, angles, and solver diagnostics are recorded for every step. Load values are automatically restored to their originals after the run.

Depends on the base power flow module. This guide covers only the time-series-specific setup. For a full explanation of `Circuit`, `Bus`, `TransmissionLine`, `Generator`, `Load`, and the Newton-Raphson solver, see [README.md](#). You should be comfortable building and solving a basic circuit before continuing here.

Prerequisites

Install dependencies and make sure the package is importable:

```
pip install numpy pandas
```

All imports in this guide assume you are running from the repository root.

Step 1 — Build the Circuit

Start by creating a `Circuit` object and adding your network equipment. You must call `calc_ybus()` before running the time-series study.

1a. Create the circuit

```
from circuit import Circuit
from bus import BusType

circuit = Circuit("My Two-Bus System")
```

1b. Add buses

Every circuit needs at least one **Slack** bus (the voltage reference) and one or more **PQ** buses (load buses).

```
# add_bus(name, nominal_kv, bus_type)
circuit.add_bus("Substation", 230.0, bus_type=BusType.Slack)
circuit.add_bus("Load Bus", 230.0, bus_type=BusType.PQ)
```

Supported bus types are `BusType.Slack`, `BusType.PQ`, and `BusType.PV`.

1c. Add a transmission line

```
# add_transmission_line(name, from_bus, to_bus, r, x)
# r and x are in per-unit on the system base
circuit.add_transmission_line("L1", "Substation", "Load Bus", r=0.0, x=0.2)
```

1d. Add a generator

```
# add_generator(name, bus, voltage_setpoint, mw_setpoint)
circuit.add_generator("G1", "Substation", voltage_setpoint=1.0, mw_setpoint=200.0)
```

1e. Add loads

Each load gets a name that you will reference later in the CSV mapping.

```
# add_load(name, bus, mw, mvar)
circuit.add_load("LD_A", "Load Bus", mw=100.0, mvar=50.0)
```

These MW and MVAR values are the **base values**. The time-series study scales them at each step.

1f. Build the Y-bus

```
circuit.calc_ybus()
```

This must be called before `TimeSeriesPowerFlow.run()` .

Step 2 — Prepare the Input CSV

The profile CSV drives the study. Each row is one time step. The file needs:

- A **step column** (e.g. `time`) that labels each row in the output. You can omit this and use the row index instead by passing `step_column=None` .
- One **modifier column per load** you want to vary. The modifier is a non-negative scale factor applied to that load's base MW and MVAR values at each step.

Modifier value	Effect
1.0	No change — load stays at its base values
0.5	Load is halved (50 MW / 25 MVAR for a 100 MW / 50 MVAR base)
1.25	Load is increased by 25 %
0.0	Load is switched off

Example — single load, five steps

Save this as `my_profile.csv` :

```
time,LD_A_scale
1,1.0
2,0.75
3,0.5
```

4,0.75
5,1.0

At step 3 the load `LD_A` would be scaled to 50 MW / 25 MVAR ($0.5 \times \text{base}$).

Example — two loads with independent profiles

```
time,LD_A_scale,LD_B_scale
1,1.0,0.8
2,0.5,1.0
3,0.75,1.2
4,1.0,0.9
```

Column names in the CSV can be anything; the mapping between column name and load name is defined in Step 4.

Step 3 — Create the Runner and Load the Profile

```
from timeseries import TimeSeriesPowerFlow

ts = TimeSeriesPowerFlow()

# load_profile(csv_path, step_column)
# step_column: the column whose values label each time step in the output.
# Pass step_column=None to use the integer row index instead.
ts.load_profile("my_profile.csv", step_column="time")
```

`load_profile` raises `FileNotFoundError` if the file does not exist and `ValueError` if `step_column` is specified but is not a column in the file.

Step 4 — Map CSV Columns to Circuit Loads

Tell the runner which CSV column controls which load in your circuit. The key is the column name in the CSV; the value is the load name as added to the circuit.

```
# Single load
ts.assign_per_load_modifiers({
    "LD_A_scale": "LD_A",
})

# Two loads with independent profiles
ts.assign_per_load_modifiers({
    "LD_A_scale": "LD_A",
    "LD_B_scale": "LD_B",
})
```

`assign_per_load_modifiers` raises `ValueError` if the mapping is empty or contains blank strings.

Step 5 — Run the Study

```
results = ts.run(circuit)
```

Optional solver parameters:

Parameter	Default	Description
ybus	None	Pass a custom Y-bus matrix; defaults to <code>circuit.y_bus</code>
tol	1e-3	Newton-Raphson convergence tolerance
max_iter	50	Maximum solver iterations per time step

`run()` raises `ValueError` if `load_profile()` or `assign_per_load_modifiers()` has not been called first. Load base values are always restored after the run completes, even if an error occurs mid-simulation.

Step 6 — Working with the Results

`run()` returns a `pandas.DataFrame` with one row per time step. The same `DataFrame` is also accessible as `ts.results` at any time after the run.

Output columns

Column	Type	Description
step	any	Value from the step column, or row index if <code>step_column=None</code>
modifier_<load_name>	float	Scale factor applied to that load at this step
V_<bus_name>_pu	float	Bus voltage magnitude in per-unit
angle_<bus_name>_deg	float	Bus voltage angle in degrees
converged	bool	<code>True</code> if the Newton-Raphson solver converged
iterations	int	Number of solver iterations used
max_mismatch	float	Final power mismatch; <code>NaN</code> if unavailable

Example output

For the two-bus circuit and five-step profile from this guide:

step	modifier_LD_A	V_Substation_pu	V_Load Bus_pu	angle_Substation_deg	angle_Load Bus_deg	converged	iterations	max_mismat
1	1.00	1.0000	0.9724	0.0	-2.87	True	4	8.1e-07
2	0.75	1.0000	0.9793	0.0	-2.15	True	4	4.9e-07
3	0.50	1.0000	0.9862	0.0	-1.43	True	3	2.0e-07
4	0.75	1.0000	0.9793	0.0	-2.15	True	4	4.9e-07
5	1.00	1.0000	0.9724	0.0	-2.87	True	4	8.1e-07

Extracting specific values

```
# All voltage magnitudes at the load bus
v_series = results["V_Load Bus_pu"]

# Voltage at a specific time step (step value 3)
v_at_step3 = results.loc[results["step"] == 3, "V_Load Bus_pu"].values[0]

# All steps where the solver did not converge
non_converged = results[results["converged"] == False]

# Summary statistics for the load modifier
print(results["modifier_LD_A"].describe())

# Select only voltage and angle columns
voltage_cols = [c for c in results.columns if c.startswith("V_")]
angle_cols = [c for c in results.columns if c.startswith("angle_")]
profile_only = results[["step"] + voltage_cols + angle_cols]
```

Step 7 — Save Results to CSV

```
ts.save_results_csv("timeseries_output.csv")
```

This writes the `results` DataFrame to a CSV file with no row index. `save_results_csv` raises `ValueError` if called before `run()`.

The saved file mirrors the DataFrame structure exactly — one header row followed by one data row per time step.

Complete Example

The following end-to-end script combines all steps above:

```
from circuit import Circuit
from bus import BusType
from timeseries import TimeSeriesPowerFlow
import pandas as pd

# Step 1 – Build the circuit
circuit = Circuit("Two-Bus Example")
circuit.add_bus("Substation", 230.0, bus_type=BusType.Slack)
circuit.add_bus("Load Bus", 230.0, bus_type=BusType.PQ)
circuit.add_transmission_line("L1", "Substation", "Load Bus", r=0.0, x=0.2)
circuit.add_generator("G1", "Substation", voltage_setpoint=1.0, mw_setpoint=200.0)
circuit.add_load("LD_A", "Load Bus", mw=100.0, mvar=50.0)
circuit.calc_ybus()

# Step 2 – Write the profile CSV
profile = pd.DataFrame({
    "time": [1, 2, 3, 4, 5],
    "LD_A_scale": [1.0, 0.75, 0.5, 0.75, 1.0],
})
profile.to_csv("my_profile.csv", index=False)

# Steps 3–5 – Configure and run
```

```
ts = TimeSeriesPowerFlow()
ts.load_profile("my_profile.csv", step_column="time")
ts.assign_per_load_modifiers({"LD_A_scale": "LD_A"})
results = ts.run(circuit)

# Step 6 – Inspect results
print(results)
print("\nLoad bus voltages:")
print(results[["step", "V_Load Bus_pu", "angle_Load Bus_deg"]])

# Step 7 – Save
ts.save_results_csv("timeseries_output.csv")
```

You can also run the equivalent built-in example script directly:

```
PYTHONPATH=. python timeseries_main.py
```

This builds the same two-bus system, writes `timeseries_input.csv` , runs the study, prints the Y-bus and full results, and saves `timeseries_output.csv` .

API Quick Reference

Method	Signature	Description
<code>load_profile</code>	<code>(csv_path, step_column="time")</code>	Read the profile CSV.
<code>assign_per_load_modifiers</code>	<code>(column_to_load: dict)</code>	Map CSV columns to load names.
<code>run</code>	<code>(circuit, ybus=None, tol=1e-3, max_iter=50)</code>	Execute the study; returns a <code>DataFrame</code> .
<code>save_results_csv</code>	<code>(output_path)</code>	Write results to CSV.
<code>ts.results</code>	attribute	The <code>DataFrame</code> returned by the last <code>run()</code> call.